

Topic 13: Exception Handling

13.1 Why Exception Handling?

- Errors that occur at runtime (file not found, division by zero, bad input) cannot be caught by the compiler
- Returning error codes mixes normal and error logic -- hard to read and easy to forget to check
- Exception handling separates error-detection code from error-handling code cleanly
- When an error occurs the program throws an exception; calling code catches it

13.2 try, throw, catch

- throw: signals that an error has occurred -- sends an exception object up the call stack
- try block: wraps the code that might throw
- catch block: handles a specific type of exception -- multiple catch blocks can handle different types
- If no catch matches, `std::terminate()` is called and the program crashes

```
#include <stdexcept>
using namespace std;

double safeDivide(double a, double b) {
    if (b == 0) throw runtime_error("Division by zero");
    return a / b;
}

int main() {
    try {
        cout << safeDivide(10, 0) << endl;
    }
    catch (const runtime_error& e) {
        cout << "Error: " << e.what() << endl;
    }
    catch (...) { // catch-all
        cout << "Unknown error" << endl;
    }
    return 0;
}
```

13.3 Standard Exception Classes

- All standard exceptions inherit from `std::exception` (in `<stdexcept>`)
- `runtime_error`: errors detectable only at runtime

- `logic_error`: errors due to incorrect program logic (`invalid_argument`, `out_of_range`)
- `bad_alloc`: thrown by `new` when heap memory is exhausted
- Call `.what()` to get the error message string

13.4 Custom Exception Classes

- Derive from `std::exception` or `std::runtime_error` to create domain-specific exceptions
- Override `what()` to provide a descriptive message

```
class InsufficientFundsException : public runtime_error {
    double shortfall;
public:
    InsufficientFundsException(double s)
        : runtime_error("Insufficient funds"), shortfall(s) {}
    double getShortfall() const { return shortfall; }
};
```

13.5 Stack Unwinding

- When an exception is thrown, C++ automatically destroys local objects in reverse order (RAII)
 - Destructors run for all stack objects between the throw and the matching catch
 - This guarantees resource cleanup even when exceptions occur -- a crucial safety mechanism
-